EXPERIMENT 07

AO* SEARCH

Artificial Intelligence / Python Practical

Experiment	07
Title	Implementation of AO* Search in Python
Language	Python
Search Type	Informed Search on AND-OR Graphs
Primary Data Structure	AND-OR graph (dictionary of option lists)

1. Aim

To write and execute a Python program that implements the AO* algorithm to find the lowest-cost solution graph in an AND-OR graph.

2. Objective

The objective is to understand how a problem that can be broken into sub-problems is represented as an AND-OR graph, and how AO* uses heuristic estimates and cost revision to find the cheapest way to solve it.

3. Theory

An AND-OR graph has two kinds of choices at a node. An OR option means that only one of the alternatives has to be solved. An AND option means that all of the listed children must be solved. The answer of AO* is therefore a solution graph, not a single path.

Each node has a cost estimate h(n). For leaf nodes it is the exact cost; for other nodes it is an estimate that may turn out to be wrong. The cost of an option is the sum of (edge cost + h(child)) over its children, and a node takes the cost of its cheapest option.

AO* expands an unsolved node on the current best partial solution graph and then revises (back-propagates) the costs of its ancestors. Because of this revision, the best option at a node can change during the search. A node is solved when all children of its best option are solved, and the search ends when the start node is solved.

4. Graph Used in the Practical

For this practical, the following AND-OR graph and cost values are used (every edge to a child costs 1):

Node	Options	h(n) / cost
A	OR: B / AND: C and D	3 (estimate)
B	AND: E and F	2 (estimate)
C	G	1 (estimate)
D	H	1 (estimate)
E	leaf	4 (exact cost)
F	leaf	3 (exact cost)
G	leaf	1 (exact cost)
H	leaf	2 (exact cost)

Start node: A

5. Algorithm

- Mark all leaf nodes as solved and set the working cost of every node to its estimate.
- While the start node is not solved, trace the current best solution graph from the start node and pick an unexpanded, unsolved node.
- Expand the node and calculate the cost of each option: sum of (edge cost + h(child)).
- Store the cheapest option as the node's best option and update its cost.
- Revise the costs and best options of the expanded nodes above it (back-propagation).
- Mark a node as solved if all children of its best option are solved.
- When the start node is solved, follow the best options from the start node to read off the solution graph and its cost.

6. Python Program

```
graph = {
    'A': [['B'], ['C', 'D']],
    'B': [['E', 'F']],
    'C': [['G']],
    'D': [['H']],
    'E': [], 'F': [], 'G': [], 'H': []
}

heuristic = {'A': 3, 'B': 2, 'C': 1, 'D': 1,
             'E': 4, 'F': 3, 'G': 1, 'H': 2}

EDGE_COST = 1

def ao_star(graph, heuristic, start):
```

```
    h = dict(heuristic)
    best = {}
    expanded = set()
    solved = {node for node in graph if not graph[node]}

    def option_cost(option):
        return sum(EDGE_COST + h[child] for child in option)

    def revise():
        changed = True
        while changed:
            changed = False
            for node in expanded:
                costs = [option_cost(o) for o in graph[node]]
                index = costs.index(min(costs))
                option = graph[node][index]
                is_solved = all(c in solved for c in option)
                if (h[node] != costs[index] or best.get(node) != option
                        or (node in solved) != is_solved):
                    h[node] = costs[index]
                    best[node] = option
                    if is_solved:
                        solved.add(node)
                    else:
                        solved.discard(node)
                    changed = True

    def find_unexpanded():
        queue = [start]
        seen = set()
        while queue:
            node = queue.pop(0)
            if node in seen or node in solved:
                continue
            seen.add(node)
            if node not in expanded:
                return node
            queue.extend(best[node])
        return None

    while start not in solved:
        node = find_unexpanded()
        if node is None:
            break
        expanded.add(node)
        revise()
```

```
    solution = {}
    queue = [start]
    while queue:
        node = queue.pop(0)
        if node in best and node not in solution:
            solution[node] = best[node]
            queue.extend(best[node])
    return solution, h[start]

solution, total_cost = ao_star(graph, heuristic, 'A')
print("AO* Solution Graph:")
for node, children in solution.items():
    print(node, "->", " AND ".join(children))
print("Total Cost:", total_cost)
```

7. Expected Output

```
AO* Solution Graph:
A -> C AND D
C -> G
D -> H
Total Cost: 7
```

8. Step-by-Step Execution

Step	Node expanded	Option costs	Result
1	A	via B = 2+1 = 3; via C AND D = (1+1)+(1+1) = 4	h(A) = 3; best option: B
2	B	E AND F = (4+1)+(3+1) = 9	h(B) = 2 → 9, B solved. A via B = 10, via C AND D = 4: best switches to C AND D
3	C	G = 1+1 = 2	h(C) = 1 → 2, C solved. h(A) = (2+1)+(1+1) = 5
4	D	H = 2+1 = 3	h(D) = 1 → 3, D solved. h(A) = (2+1)+(3+1) = 7, A solved

At step 1 the estimate for B (2) makes the option via B look cheapest. Expanding B shows that its real cost is 9, so the revised cost of A through B becomes 10 and AO* switches to the AND option (C and D). The final solution graph is A → (C AND D), C → G, D → H, with a total cost of 7.

9. Time and Space Complexity

The running time depends on the structure of the AND-OR graph and on the quality of the heuristic. In this implementation every node is expanded at most once, and each expansion is followed by a revision pass over the expanded nodes, giving a simple upper bound of $O(V^2 \cdot E)$, where V is the number of nodes and E the number of edges. The space requirement is O(V) for the cost estimates, best options and the expanded and solved sets.

10. Advantages

- Handles problems that can be decomposed into sub-problems (AND arcs), which A* cannot represent.
- Uses heuristic estimates to avoid expanding nodes that are not on a promising solution graph.
- Revises its choices when an estimate turns out to be wrong.
- Finds an optimal solution graph when the heuristic never overestimates.

11. Disadvantages

- More complex to implement than A* or Best First Search.
- Needs a good, non-overestimating heuristic for optimal results.
- Cost revision adds extra work after every expansion.
- Standard AO* assumes the graph has no cycles.
- Memory use can be large for big AND-OR graphs.

12. Applications

- Problem decomposition and planning
- Game playing and puzzle solving
- Expert systems and diagnosis
- Theorem proving
- Task scheduling with sub-tasks

13. AO* vs A*

Feature	A*	AO*
Graph type	OR graph (ordinary graph)	AND-OR graph
Answer	A single path	A solution graph
Cost of a choice	g(n) + h(n)	Sum of (edge cost + h(child)) over an option
Cost revision	Not needed	Back-propagates revised costs to ancestors
AND arcs	Not supported	Supported

14. Viva Questions and Answers

1. What is AO* search?

It is an informed search algorithm that finds the lowest-cost solution graph in an AND-OR graph.

2. What is an AND-OR graph?

A graph where a node can be solved by one of several alternatives (OR) or by solving several sub-problems together (AND).

3. What is the difference between an AND arc and an OR arc?

An OR arc needs only one child to be solved; an AND arc needs all of its children to be solved.

4. What is a solution graph?

The set of nodes and chosen options that together solve the start node. It is the answer of AO*, instead of a single path.

5. What is cost revision in AO*?

After a node is expanded, its new cost is passed up to its ancestors, which may then change their best option.

6. How is the cost of an AND option calculated?

By summing (edge cost + cost of the child) over all children of that option.

7. What is the difference between A* and AO*?

A* searches an ordinary graph for a path using g(n) + h(n); AO* searches an AND-OR graph for a solution graph using cost revision.

8. When is AO* optimal?

When the heuristic never overestimates the true cost of a node.

9. When does AO* stop?

When the start node is marked solved.

15. Conclusion

AO* was implemented in Python for an AND-OR graph using heuristic estimates and cost revision. The experiment shows how an optimistic estimate can mislead the search at first and how revision corrects it: AO* switched from the option via B to the AND option (C and D) and found a solution graph of total cost 7.

16. Practical Record Note

This document is a clean, portfolio-ready practical record template. If your college-issued practical sheet uses a different graph, program, or output, those source-specific details should replace the corresponding sections before submission.